communitycore 使用指南

> 版本：当前主干 | 协议：Protobuf 二进制 + JSON 错误 | 测试：70 个测试全部通过

---

1. 概述

`communitycore` 是一个基于 Shelf 框架的 Dart 后端插件，为宿主应用提供完整的社区功能 HTTP API。它包含文章、评论、点赞、关注、分类、媒体、阅读记录与内容审核等八大模块，开箱即用，只需在宿主 `Router` 上调用一行代码即可完成接入。

技术栈

依赖	说明
`shelf` / `shelf_router`	HTTP 层与路由
`protobuf`	请求体与响应体序列化
`dart_jsonwebtoken`	JWT 身份认证
`wxtmysql`	MySQL 数据库访问（GitHub）

---

2. 功能模块介绍

2.1 模块总览

模块	路由数	功能
文章（Article）	5	发布、编辑、删除、查询列表、查询详情
评论（Comment）	4	发表、编辑、删除、查询
点赞（Like）	6	文章点赞/取消/查询，评论点赞/取消/查询
媒体（Media）	3	添加、删除、查询附件
分类（Category）	1	查询全部分类（含多语言翻译）
关注（Follow）	3	关注用户、取消关注、查询关注列表
阅读记录（Read）	2	记录阅读、查询阅读历史
内容审核（Moderation）	7	黑名单管理、屏蔽分类、举报文章、不感兴趣

2.2 完整路由列表

```
# 文章
POST /api/createArticle
POST /api/modifyArticle
POST /api/deleteArticle
POST /api/getArticles
POST /api/getArticleDetail

# 评论
POST /api/addComment
POST /api/deleteComment
POST /api/modifyComment
POST /api/getComments

# 点赞
POST /api/likeArticle
POST /api/cancelLikeArticle
POST /api/getArticleLikes
POST /api/likeComment
POST /api/canceLikeComment      ← 注意拼写（历史命名）
POST /api/getCommentLikes

# 媒体
POST /api/addMedia
POST /api/deleteMedia
POST /api/getMedias

# 分类
POST /api/getCategory

# 关注
POST /api/followUser
POST /api/unfollowUser
POST /api/getFollow

# 阅读记录
POST /api/readArticle
POST /api/getArticleReads

# 内容审核
POST /api/addBlacklist
POST /api/removeBlacklist
POST /api/getUserBlacklist
POST /api/addBlockedCategory
POST /api/removeBlockedCategory
POST /api/getUserBlockedCategories
POST /api/addReportedArticle
POST /api/getUserReportedArticles
POST /api/addDislikedArticle
POST /api/getUserDislikedArticles
```

2.3 文章列表（`getArticles`）分类枚举

`GetArticlesRequest.classfication` 字段取值：

值	语义
1	推荐（recommended）
2	发现（discover）
3	关注流（follow）
4	我的发布（myPublished）
5	草稿箱（myDraft）
6	我的点赞（myLike）
7	我的阅读（myRead）
8	我的举报（myReport）

> 分类过滤：在查询推荐/发现流时，可同时设置 `GetArticlesRequest.CategoryID` 字段（ `int32` ）按分类筛选。 `CategoryID = 0` 表示不过滤。

---

3. 架构设计

3.1 整体分层

```
宿主 Router
    │
    └── setupCommunityRoutes()          ← 单一接入点
            │
            ├── communityErrorMiddleware()     ← 全局异常捕获，统一返回 JSON
            ├── communityAuthMiddleware()      ← JWT 解析，注入 uid 至请求
            │
            ├── ArticleRoutes
            ├── CommentRoutes
            ├── LikeRoutes
            ├── MediaRoutes
            ├── CategoryRoutes
            ├── FollowRoutes
            ├── ReadRoutes
            └── ModerationRoutes
                    │
                    └── CommunityRepository  ← 统一数据库访问层
                            │
                            └── DatabaseService.instance  ← wxtmysql
```

3.2 中间件管道

所有路由均经过以下两层中间件（顺序固定）：

1. **`communityErrorMiddleware`**：捕获路由中抛出的所有未处理异常，返回 `{"code": 500, "msg": "..."}` JSON。
2. **`communityAuthMiddleware`**：验证请求头中的 JWT Token，解析 `uid` 并写入 `Request` 上下文。认证失败直接返回 `401` 。

3.3 通信协议

场景	Content-Type	格式
请求体	`application/x-protobuf`	Protobuf 二进制
正常响应	`application/x-protobuf`	Protobuf 二进制
参数错误（400）	`application/json`	`{"code": 2, "msg": "..."}`
Content-Type 错误（400）	`application/json`	`{"code": 1, "msg": "..."}`
资源不存在（404）	`application/json`	`{"code": 404, "msg": "..."}`
服务端异常（500）	`application/json`	`{"code": 500, "msg": "..."}`

> 所有请求体必须携带 `Content-Type: application/x-protobuf` 或 `application/octet-stream` ，否则返回 `code=1` 的 400 错误。

3.4 数据库表结构

communitycore 自有表均以 `cm_` 为前缀，由 `setupCommunityRoutes` 启动时自动创建（ `IF NOT EXISTS` ）。共享表（ `users` 、 `Friendships` ）由其他插件管理，communitycore 仅做只读 JOIN。

常量类	对应数据库表	说明
`ArticleTable`	`cm_Article`	文章主体，含状态、分类、隐私等字段
`ArticleMediaTable`	`cm_ArticleMedia`	文章附件（图片/视频 URL）
`ArticleCommentTable`	`cm_AComment`	评论，支持嵌套回复（ `ParentCommentID` ）
`ArticleLikeTable`	`cm_ArticleLike`	文章点赞记录
`CommentLikeTable`	`cm_CommentLike`	评论点赞记录
`ArticleReadTable`	`cm_ArticleRead`	阅读历史

`FollowTable`	`cm_Follow`	关注关系（follower → followed）
`CategoryTable`	`cm_Category`	分类基础信息
`CategoryTranslationTable`	`cm_CategoryTranslation`	分类多语言翻译
`BlacklistTable`	`cm_Blacklist`	用户黑名单
`BlockedCategoryTable`	`cm_BlockedCategory`	屏蔽分类
`ReportedArticleTable`	`cm_ReportedArticle`	举报记录
`DislikedArticleTable`	`cm_DislikedArticle`	不感兴趣记录
`UserTable`	`users`	⚠️ 共享表（authsql 所有），只读 JOIN 获取作者信息
—	`Friendships`	⚠️ 共享表（chatcore 所有），只读 JOIN 获取好友关系

3.5 CommunityNotifier 扩展点

文章状态变更（投稿 / 审核中 / 已发布 / 拒绝）和文章被举报时，插件会调用 `CommunityNotifier` 回调，宿主应用可通过注入自定义实现接入消息推送系统。

```
abstract CommunityNotifier
    │
    ├── NoOpCommunityNotifier    ← 默认实现（const，全部静默）
    └── CommunityNotifierBase    ← 带消息推送能力的基类
            │
            └── 宿主自定义类（实现 sendSystemMsg）
```

---

4. 快速接入

4.1 添加依赖

在宿主应用的 `pubspec.yaml` 中添加本地路径依赖（`communitycore` 本身通过 Git 引入 `wxtmysql`，无需额外声明）：

```
dependencies:
  communitycore:
    path: ../plugins/communitycore
```

`communitycore` 的 `wxtmysql` 依赖来自 Git 远程仓库，已在插件内 `pubspec.yaml` 中声明：

```
# communitycore/pubspec.yaml
dependencies:
  wxtmysql:
```

```
    git:
      url: https://github.com/SheltonWan/wxtmysql.git
      ref: main
```

4.2 最小接入示例

```
import 'package:shelf_router/shelf_router.dart';
import 'package:communitycore/communitycore.dart';

final router = Router();

// setupCommunityRoutes 为异步方法，务必 await
await router.setupCommunityRoutes(
  jwtSecret: 'your-jwt-secret',
);
```

> 自动建表：`setupCommunityRoutes` 内部会在启动时调用 `CommunityRepository.createTables()`，以 `IF NOT EXISTS` 幂等方式创建全部 13 张 communitycore 数据表，无需手动执行任何 SQL 或建表脚本。

4.3 注入自定义通知器

```
await router.setupCommunityRoutes(
  jwtSecret: 'your-jwt-secret',
  notifier: MyAppNotifier(),
);
```

---

5. CommunityNotifier 自定义指南

5.1 方式一：直接实现接口（完全自定义）

```
class MyAppNotifier implements CommunityNotifier {
  @override
  void notifyArticleStatusChanged(
    String status,
    Map<String, dynamic> articleRow,
    CommunityRepository repo,
  ) {
    // 自行处理文章状态变更通知
    print('文章状态变更：$status');
  }

  @override
  void notifyArticleReported(
    Map<String, dynamic> articleRow,
    CommunityRepository repo,
    String reason,
  ) {
    // 自行处理举报通知
```

```
    print('文章被举报，原因：$reason');
  }
}
```

5.2 方式二：继承 CommunityNotifierBase（推荐）

`CommunityNotifierBase` 已实现消息内容的组装逻辑，只需覆盖 `sendSystemMsg` 接入消息推送即可：

```
class ChatManagerNotifier extends CommunityNotifierBase {
  final ChatManager chatManager;
  ChatManagerNotifier(this.chatManager);

  @override
  void sendSystemMsg(
    String content,
    String? recipientId,    // null 表示发给管理员
    Map<String, dynamic> other,
  ) {
    chatManager.sendSystem(
      content: content,
      toUserId: recipientId,
      extra: other,
    );
  }
}
```

`notifyArticleStatusChanged` 触发场景与消息类型：

触发时机	`otherType` 值	`recipientId`
用户投稿	`submitArticle`	`null`（管理员）
进入审核	`articleUnderReview`	作者 uid
审核通过	`articlePublished`	作者 uid
审核拒绝	`articleRejected`	作者 uid
文章被举报	`report`	作者 uid + `null`（管理员）

---

6. 请求/响应示例

6.1 获取文章列表

请求

```
POST /api/getArticles
Content-Type: application/x-protobuf
Authorization: Bearer <JWT>
```

```
[GetArticlesRequest protobuf binary]
```

`GetArticlesRequest` 关键字段：

字段	类型	说明
`page`	`int32`	页码（从 1 开始）
`pageSize`	`int32`	每页数量
`classfication`	`int32`	文章分类枚举（见 2.3 节）
`CategoryID`	`int32`	按分类 ID 过滤（0 = 不过滤）

成功响应

```
200 OK
Content-Type: application/x-protobuf

[GetArticlesResponse protobuf binary]
```

`GetArticlesResponse` 包含 `repeated Article articles` 列表，关注流（`classfication=3`）的每个 `Article` 会自动附带 `CommunityUser author` 字段（含 `uid`、`username`、`faceUrl`）。

6.2 错误响应

```
// 400 - 参数缺失或非法
{"code": 2, "msg": "Parameter error"}

// 400 - Content-Type 不正确
{"code": 1, "msg": "Content-Type must be application/x-protobuf"}

// 404 - 资源不存在
{"code": 404, "msg": "Not found"}

// 500 - 服务端异常
{"code": 500, "msg": "Internal server error"}
```

---

7. 测试指南

7.1 运行测试

```
cd backend/plugins/communitycore
dart test
```

测试套件包含 **70 个 unit tests**，均使用 `FakeCommunityRepository` 替代真实数据库。

7.2 使用 FakeCommunityRepository 编写测试

```
import 'package:communitycore/src/repositories/
fake_community_repository.dart';
import 'package:test/test.dart';
import 'package:shelf/shelf.dart';
import 'package:shelf_router/shelf_router.dart';

void main() {
  late FakeCommunityRepository fakeRepo;
  late Router router;

  setUp(() {
    fakeRepo = FakeCommunityRepository();
    router = Router();
    // 直接调用路由注册方法，传入 fake repo
    setupArticleRoutes(router, fakeRepo, const NoOpCommunityNotifier());
  });

  test('创建文章', () async {
    final request = Request('POST', Uri.parse('http://localhost/api/
createArticle'),
      body: createArticleRequest.writeToBuffer(),
      headers: {'content-type': 'application/x-protobuf', 'uid': '42'},
    );
    final response = await router(request);
    expect(response.statusCode, equals(200));
  });
}
```

> `FakeCommunityRepository` 使用内存列表模拟所有数据库操作，支持所有 CRUD 方法，无需数据库连接即可完整测试业务逻辑。

---

8. 常见问题

Q：请求返回 `code=1` 的 400 错误？

A：请求头未正确设置 `Content-Type: application/x-protobuf` 。确保客户端（如 Dio）在发送 protobuf 请求时携带此 header。

Q：请求返回 401？

A：JWT Token 无效或已过期。检查 `Authorization: Bearer <token>` header 与服务端 `jwtSecret` 是否匹配。

Q：关注流没有返回作者信息？

A：确认 proto 已是最新版本（ `Article` message 包含 `CommunityUser author = 15` 字段），前后端 pb.dart 均需重新生成。

Q：如何按分类过滤？

A：在 `GetArticlesRequest` 中同时设置 `classfication` （推荐=1 或发现=2）和 `CategoryID` （非零整数）。Follow 流（ `classfication=3` ）不支持分类过滤。